Flocking and Collective Behavior: When Many Become One

Joshua S. Weitz[1, 2, *]

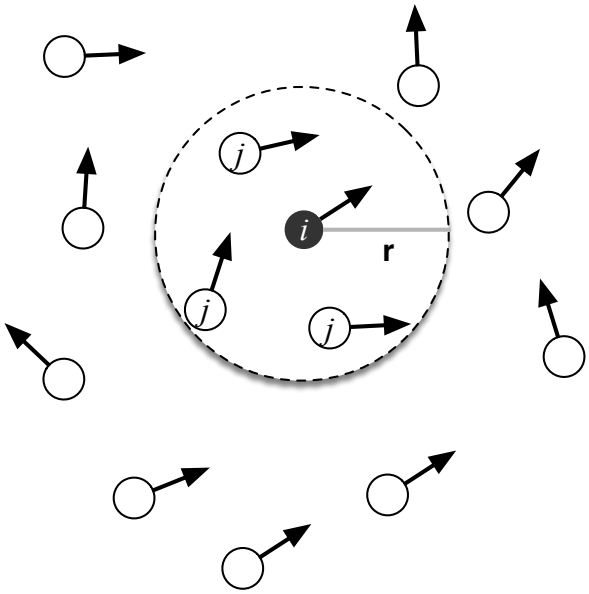
[1] *School of Biology, Georgia Institute of Technology, Atlanta, GA, USA*
[2] *School of Physics, Georgia Institute of Technology, Atlanta, GA, USA*

(Dated: November 13, 2020)

I. AGENT-BASED MODELS AND EMERGENCE IN FLOCKS

Birds of a feather, flock together. This old adage reminds us of certain apparent 'true-isms' – both good and bad – that characterize communities. There are advantages to sticking together, whether the groups are insects, fish, or primates. These advantages arise from pro-social behaviors, including defense, feeding, mating, and yes, even play. Yet organisms who are in motion don't just have to stick together, like birds on a wire in the words of Leonard Cohen's classic folk tune, but they also have to move together. Moving together is a form of emergent choreography. And, until the mid 1990s, it was quite unclear exactly how organisms could accomplish such a 'group'-level trait. It was generally understood that when a group of starling pigeons moves, nearly fluid-like, in the evening sky, there is not a gene in each that encodes that particular set of movements, at the particular time and place, and specifies each bird's particular position in the flock. That seems obvious. But, if such flock movements are not prescribed, then it points to something challenging: what are the local rules of interaction that govern movement and how can local rules lead to emergent population-level dynamics?

Answering this question forms the basis for this laboratory. which explores the Vicsek model of flocking behavior. The Vicsek model or the 'self-propelled particle' (SPP) model was proposed in the mid 1990s. The core idea is revealed in the following figure:



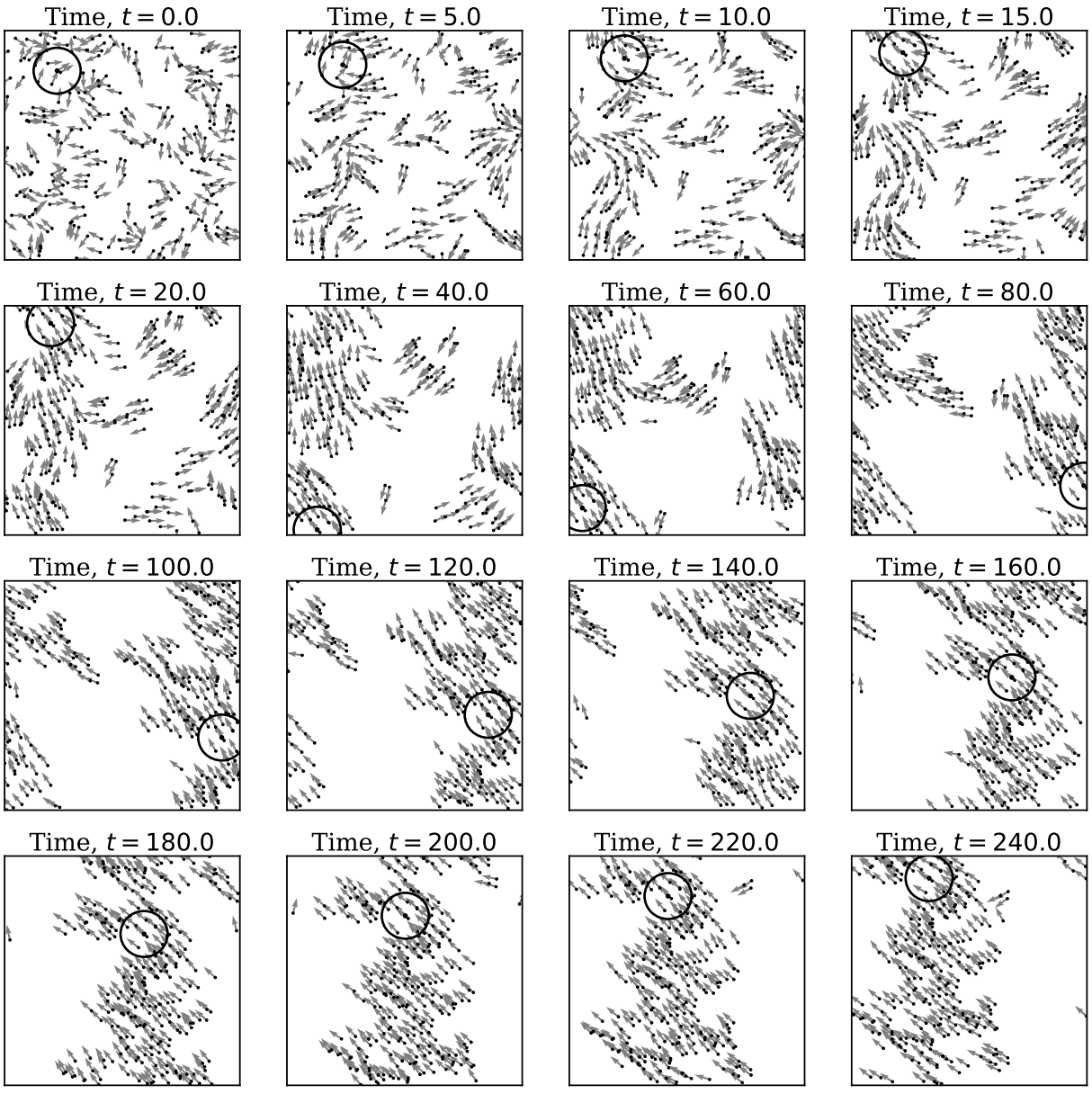
The image depicts a focal individual (closed circle) surrounded by other individuals (open circles); all of the individuals are moving, but not necessarily in the same direction. The focal individual i senses the movements of organisms close to it. In this example the term 'close' applies to all those individuals j whose current position is within r of individual i. The model then implements a rule: that the direction in which a focal organism moves is a function of its current velocity, v_i, the velocity of its 'close' neighbors $\{v_j\}$, as well as some element of randomness. If this same rule applies to all particles, then it is possible for this local rule to translate into including emergent dynamics seemingly quite unexpected and unrelated to the local rule.

---

*Electronic address: jsweitz@gatech.edu; URL: http://ecotheory.biology.gatech.edu

Typeset by REVTEX

This lab provides a basis for moving from local rules, as described by SPP theory, but you will be expected to implement the entire code. The code then becomes the basis for exploring dynamics as parameters vary, with a particular emphasis on the transition from individual-like behavior to flock-like behavior. As it turns out, this transition has all the properties of phase transitions in the study of physical materials (water to ice) and in the study of static, complex systems (as in percolation theory or connectivity of some, and then suddenly, to all agents in a system). Here, a notable difference is that there is a dynamic order parameter associated with very long length scales that approach that of the system size in which the behavior appears coherent and (nearly) identical. The snapshots below illustrates this point. Initially, a collection of 300 individuals moves at random (at $t = 1$). This, given feedback via local rules, these individuals spontaneous exhibit large-scale older and flocks (sometimes spanning nearly the entire population) moving together, even if no one organism has any preference regarding where or why to move in a particular direction. As a guide to the eye, pay attention to the focal individual, arbitrarily surrounded by a black circle – even if it is not moving with the flock at the start, it soon is, along with all the other individuals. This form of spontaneous symmetry breaking and emergent order is the centerpiece of this computational laboratory.



II. VICSEK MODEL

A. Core model

The Vicsek or SPP model simulates the dynamics of flocking organisms or "boids". The emergence of flocking requires that local information is transmitted to groups and potentially to the entire population. This transmission is not encoded directly in the model, but is a potential outcome, given suitable conditions. At each time point, the direction of motion for each boid is determined from both the average direction of neighboring boids within a radius, $r = 1$, and a noise term, η. We will follow the original paper and implement the model in a 2D setting with periodic boundary conditions. The pseudocode for the SPP model is as follows:

- Initialize the position $(x_i(t), y_i(t)$ and direction $\theta_i(t)$ of each boid – note the model maintains a constant speed of each boid while changes its direction.
- Update the direction of every boid's movement, $\theta_i(t+1)$, by reorienting based on the local direction of neighbors.

$$\theta_i(t+1) = \langle \theta_j(t) \rangle_R = \arctan \frac{\langle \sin \theta_j(t) \rangle_R}{\langle \cos \theta_j(t) \rangle_R} + \pi \Theta(-\langle \cos \theta_j(t) \rangle_R) + noise$$

- Update the position of each boid i assuming ballistic motion.

$$x_i(t+1) = x_i(t) + v_0 \Delta t \cos(\theta_i(t+1))$$
$$y_i(t+1) = y_i(t) + v_0 \Delta t \sin(\theta_i(t+1))$$

That's the entirety of the model. However, the challenge lies in the updating component and represents the first challenge problem.

Challenge Problem: Reorienting Boids

Write a code to reorient boids based on the orientation of neighbors, i.e., including both an averaging component as well as a random term. *Hint: identify a solution to the problem that arises in defining a local 'group' if the focal boid is alone..*

Solutions to Challenge Problem: Reorienting Boids In the SPP model, the direction is updated based on the orientation of neighboring boids.

$$\theta_i(t+1) = \langle \theta_j(t) \rangle_R = \arctan \frac{\langle \sin \theta_j(t) \rangle_R}{\langle \cos \theta_j(t) \rangle_R} + \pi \Theta(-\langle \cos \theta_j(t) \rangle_R) + noise$$

where the averages $\langle * \rangle_R$ are over the indices of j corresponding to the boids within R of the focal boid indexed by i. The numerator in the argument of the arctan is the average y-direction of the neighboring boid velocities, and the denominator is the average x-direction of the neighboring boid velocities. By taking the arctan of the ratio we get a new angle. However, the range of the arctan operator is between $[\frac{-\pi}{2}, \frac{\pi}{2}]$, thus if the velocity in the x-direction is less than 0, we must correct by shifting by π. This is the second term, where we utilize the Heaviside function, $\Theta(x)$, which is 1 if $x > 0$ and 0 otherwise (note the minus sign in the argument of the Heaviside function in the main equation). Consider different combinations of the signs of the x and y component of the velocity and determine the appropriate correction term. Lastly, the noise term is uniformly sampled random numbers between $[-\frac{\eta}{2}, \frac{\eta}{2}]$.

This model can be implemented as follows assuming that `currx` and `curry` denote the positions of all boids and `currtheta` denote the current angle. In this code, the noise is scaled between $-\eta$ and η, and this enables modulation to favor local averaging. The key idea is to calculate the local distances using periodic boundary conditions (hence the use of the `min` distance metric).

```
# Loop i through each boid
for i in range(N):
    # Find distances
    dx = np.minimum(np.abs(currx-currx[i]),
                    np.abs(currx-(currx[i]+L)),
                    np.abs(currx-(currx[i]-L)))
    dy = np.minimum(np.abs(curry-curry[i]),
                    np.abs(curry-(curry[i]+L)),
                    np.abs(curry-(curry[i]-L)))
    boid_r2 = np.sqrt( (dx)**2 + dy**2 )
    #Identify the set of local boids
    avgthetas = currtheta[boid_r2<=r]
    #Find the orientation of local boids
    avgx = np.mean(np.cos(avgthetas))
    avgy = np.mean(np.sin(avgthetas))
    # Reset the new direction
    newtheta[i] = np.arctan2(avgy,avgx)
deltheta = -eta/2+eta*np.random.uniform(size=N)
currtheta = newtheta+deltheta
```

B. Numerical implementation

When fully integrated, the main code tracks the x and y positions of each boid along with the orientation θ of motion for a set number of time steps, T. The simulation begins by randomly placing each boid in the domain. Additionally, the initial velocity of each boid is oriented in a random direction, θ between $[0, 2\pi]$. It is now your turn to build your SPP model.

Challenge problem: SPP Simulation Code

Set-up the main code to initialize the dynamics and loop through time. A skeleton for the code is as follows. The input parameters to this function are the total number of boids, N, the linear dimension of the domain, L, and the amount of noise, η. *Hint: implement the dynamics with periodic boundary conditions, but keep this in mind while trying to detect which boids are close to each other.*

```
def boids_dyn(pars):
    '''
    function [xt,yt,thetat]=boids_dyn(N,L,eta)
    inputs:
    N--number of boids, L--linear dimension of domain, eta--strength of noise
    outputs
    xt and yt -- positions of each boid over time
    thetat -- direction of velocity of each boid over time
    '''
    #simulation parameters
    totsteps = 1000
    delt = 1
    v0 = 0.03
    R = 1

    #initial conditions
    currx = L*np.random.uniform(size=N)
    curry = L*np.random.uniform(size=N)
    currtheta = 2*np.pi*np.random.uniform(size=N)

    #preallocate output
    xt = np.zeros((totsteps,N))
    yt = np.zeros((totsteps,N))
    thetat = np.zeros((totsteps,N))

    #run the sim (fill this in)
    # Loop t through time steps

    # Loop i through each boid

    # find the boids within R of i

    # calculate the new direction of boid i based on the directions of neighbors

    # end Loop i

    return [xt,yt,thetat]
```

Solutions to Challenge Problem: SPP Simulation Code.
The final code is below. A critical set of lines is how to find those particles within r of the focal boid A 'naive' choice of squared distances does not work in the periodic boundary conditions. Hence, one must first find the minimum distance in both the x and y directions before selecting the subset. The other challenge is to reposition boids in the domain between 0 to L if they move outside. This simulation assumes that the boids are moving in a 2D domain with periodic boundary conditions to reduce finite size effects. However, alternatives can also be imposed such that there are hard boundary conditions.

```
def boids_dyn(pars):
    '''
    function [xt,yt,thetat]=boids_dyn(N,L,eta)
    inputs:
    N--number of boids, L--linear dimension of domain, eta--strength of noise

    outputs
    xt and yt -- positions of each boid over time
    thetat -- direction of velocity of each boid over time
    '''

    #simulation parameters
    N=pars['N']
    eta=pars['eta']
    L=pars['L']
    totsteps=pars['totsteps']
    delt=pars['dt']
    v0=pars['v0']
    r=pars['r']

    #initial conditions
    currx = L*np.random.uniform(size=N)
    curry = L*np.random.uniform(size=N)
    currtheta = 2*np.pi*np.random.uniform(size=N)

    #preallocate output
    xt = np.zeros((totsteps,N))
    yt = np.zeros((totsteps,N))
    thetat = np.zeros((totsteps,N))

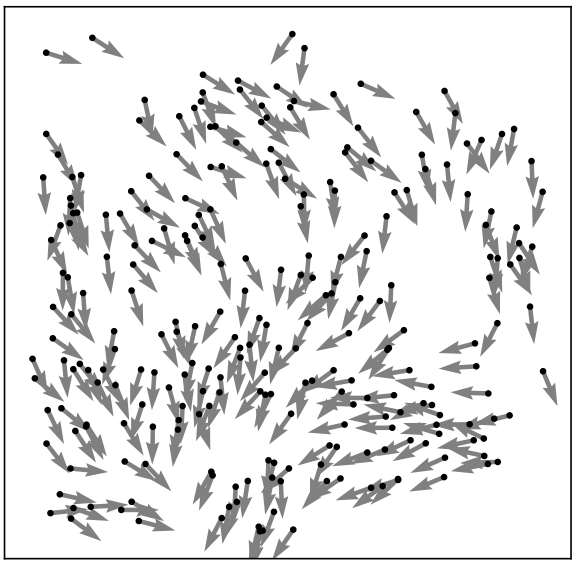
    #run the sim (fill this in)
    # Loop t through time steps
    for t in range(totsteps):
        #Preallocate new directions
        newtheta = np.zeros(N)
        # Loop i through each boid and determine new directions
        for i in range(N):
            # find the boids within R of i
            dx = np.minimum(np.abs(currx-currx[i]),
                            np.abs(currx-(currx[i]+L)),
                            np.abs(currx-(currx[i]-L)))
            dy = np.minimum(np.abs(curry-curry[i]),
                            np.abs(curry-(curry[i]+L)),
                            np.abs(curry-(curry[i]-L)))
            boid_r2 = np.sqrt( (dx)**2 + dy**2 )
            #calculate the new direction of boid i based on the directions of neighbors
            avgthetas = currtheta[boid_r2<=r]
            #average x and y components of vectors
            avgx = np.mean(np.cos(avgthetas))
            avgy = np.mean(np.sin(avgthetas))
            #obtain average direction from arctan, correcting when x direction is negative
            newtheta[i] = np.arctan2(avgy,avgx)
            # end Loop i
        #obtain vector of noise to apply to boid directions
        deltheta = -eta/2+eta*np.random.uniform(size=N)
        #update directions
        currtheta = newtheta+deltheta
        #update positions based on ballistic motion and new directions
        currx = np.mod(currx + v0*np.cos(currtheta)*delt, L)
        curry = np.mod(curry + v0*np.sin(currtheta)*delt, L)
```

C. Visualization basics

The visualization of boids can be accomplished in multiple ways. However, to represent both the position and velocity of boids, it can be useful to implement a visualization that goes beyond merely showing where the organisms are but also visualizes where they are going. For example, if the variables are `xt`, `yt`, `vxt`, `vyt` represent x, y, v_x, and v_y respectively, it is possible to use the following command for visualization:

```
plt.quiver(xt,yt,vxt,vyt,color='grey')
plt.plot(xt,yt,'k.',linestyle='')
```

The `quiver` command can be modulated with optional parameters and the second command places a circle where the boid is. Here is a snapshot of example dynamics. Notice how the direction of boid movement can be different even if the boids are near each other. The high noise causes deviations from the ordering.



At this point you are ready to explore the transition from individual to collective motion in the SPP model framework.

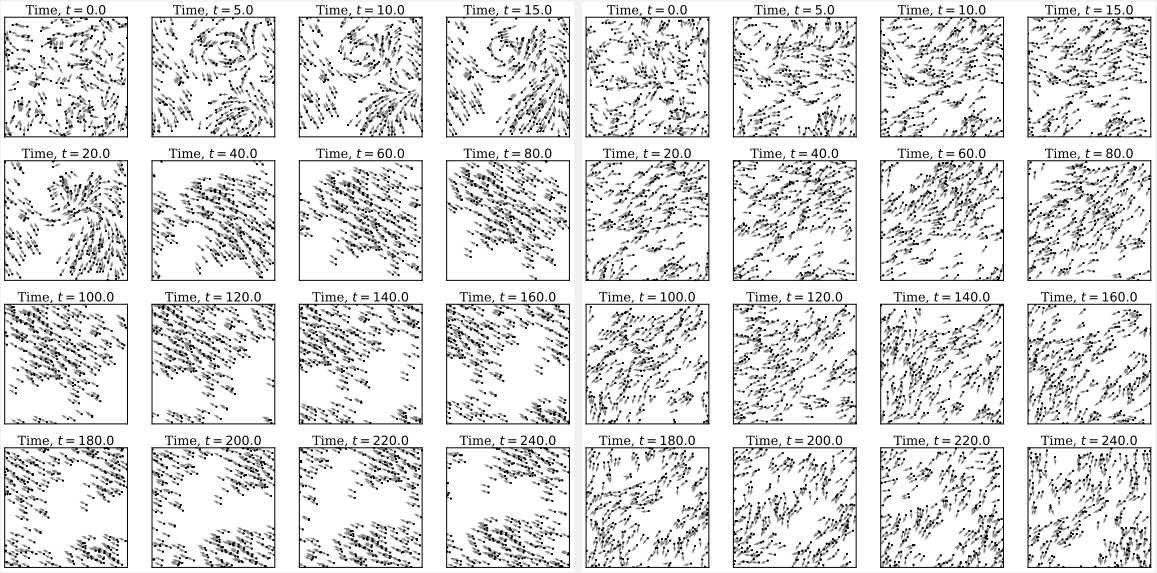
III. FLOCKING DYNAMICS

A. Baseline case

The dynamics of boids in the SPP model can be recapitulated in two regimes, the low and the high noise regimes. Here, we will graphically explore the dynamics of the Vicsek model. The first way is to simply explore a series of images taken from the initial configuration to $t = 1200$ in a boid simulation with the following parameters: $N = 250$, $L = 5$, corresponding to a density of 10 boids per unit square, $v_0 = 0.03$, dt $= 0.2$. Next explore the low-noise case where $r = 0.5$ and $\eta = 0.1$ and the high-noise case where $r = 0.05$ and $\eta = 1$. The outcomes are stark. Hence, the first challenge problem is to recapitulate the dynamics and develop a plotting code for these SPP flipbooks.

Challenge Problem: Visualizing the Flock

Develop a visualization code that generates a panel of 16 images given a specified time set and positions and velocities of each boid. Use this code to visualize the low- (left) and high-noise (right) cases. If your code works it should look (nearly) like this:



Solutions to Challenge Problem: Visualizing the Flock.

The visualization of a dynamic flock requires conveying a sense of time. The most direct way is to use a movie, i.e., by capturing multiple stills and then visualizing them together. But, this is a book, and so connected still images can also serve an important rule. This is the solution presented in the code below, the choice of a 4×4 grid is arbitrary. It is also possible to construct your own axes, but this takes more time. To switch from flipbooks to video, simply focus on one plot and replace the quivers so you can see a flock in action.

```
nstrip = 16
tmpt=np.concatenate(([0,25,50,75],np.arange(100,1201,100)))
fig = plt.figure(figsize=(24,24))
for i in range(nstrip):
    ax = plt.subplot(4,4,i+1)
    plt.plot(xt[tmpt[i],:],yt[tmpt[i],:],'k.',linestyle='')
    plt.quiver(xt[tmpt[i],:],yt[tmpt[i],:],
               np.cos(thetat[tmpt[i],:]),np.sin(thetat[tmpt[i],:]),
               color='grey',scale=15,width=0.008)
    plt.title('Time, $t={time}$'.format(time=tmpt[i]*pars['dt']),fontsize=20)
    plt.xlim([0,pars['L']])
    plt.ylim([0,pars['L']])
    plt.yticks([])
    plt.xticks([])
```

Challenge Problem: Flock of Boids – The Movie

Building upon the prior code, develop a movie version of your flock in a single panel, then watch it. It will be particularly appealing if you use $\eta = 0.25$, $r = 0.5$, and $N = 500$.

Solutions to Challenge Problem: Flock of Boids – The Movie.
The following function `movie_dyn` uses the same plot commands as above and takes as input the position and angles to plot each of the boids as they move in time and space. This is not a complicated code, but it is helpful, and can allow you to watch the collective in a continuous, aka movie, mode. The `pause` command is useful for setting the effective frame rate.

```
def movie_dyn(xt,yt,thetat,pars):
    fig = plt.figure(figsize=(5,5))
    ax = fig.gca()
    for i in range(len(xt)):
        ax.clear()
        plt.plot(xt[i,:],yt[i,:],'k.',linestyle='')
        plt.quiver(xt[i,:],yt[i,:],
               np.cos(thetat[i,:]),np.sin(thetat[i,:]),
               color='grey',scale=15,width=0.008)
        plt.xlim(0,pars['L'])
        plt.ylim(0,pars['L'])
        plt.xticks([])
        plt.yticks([])
        plt.pause(0.01)
    return fig
```

B. Varying noise

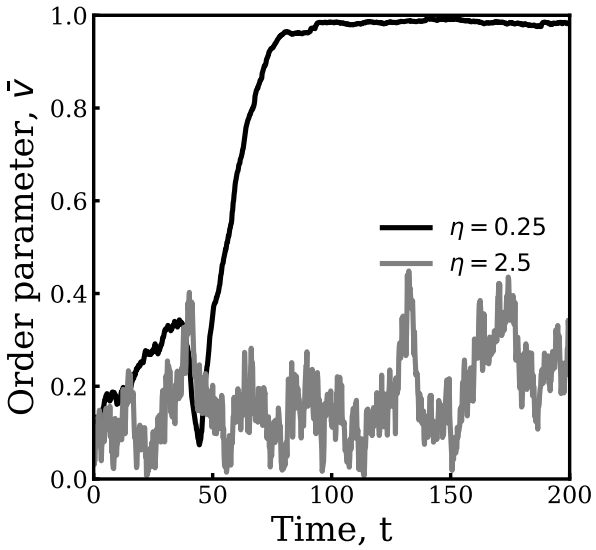
The parameter η controls the amount of noise in the model. As $\eta \to \pi$, each boid essentially moves in a random direction between each move. In doing so, increasing noise reduces the correlation of angles between neighboring boids. Quantifying the ordering requires an 'order parameter', i.e., a feature that describes the macroscopic behavior of the system rather than the behavior of an individual or small set of individuals. However, when many behave like one, then the order parameter can change rapidly from 0 (disordered, individuals acting distinctly) to 1 (ordered, individuals acting as a flock). The order parameter we will focus on is the average velocity in the SPP model:

$$\bar{v} = \frac{1}{Nv_0}\left|\sum_i^N \vec{v}_i\right|$$

The average velocity is equal to the sum of velocity vectors divided by the sum of the magnitude of each vector. An order parameter near 0 means the boids are not correlated in direction and an order parameter near 1 means all the boids face nearly the same direction. Calculate the order parameter across time. The following function calculates the order parameter:

```
def dyn2orderpar(thetat):
#Calculate the average velocity - order parameter
    N=len(thetat[0,:])
    vxt=np.cos(thetat)
    vyt=np.sin(thetat)
    sumvxt=np.sum(vxt,axis=1)
    sumvyt=np.sum(vyt,axis=1)
    normvt=np.sqrt((sumvxt)**2+(sumvyt)**2)/N
    return normvt
```

The following examples show the dynamics of $\bar{v}$ for both a disordered state and an ordered state using $N = 250$, $L = 5$, $r = 0.3$, $\eta = 0.25$ (for the ordered state) and $\eta = 2.5$ (for the disordered state) as simulated over 1000 time steps:

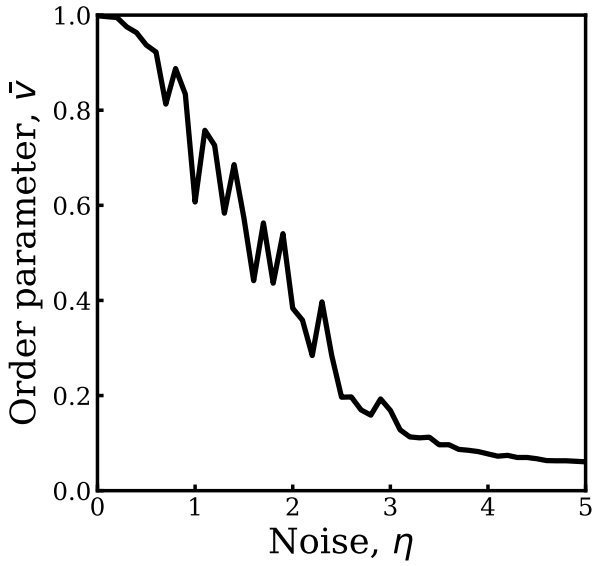


This illustration provides a sense that η can act to modify the degree of order in the system, in concert with other parameters. It is worthwhile to explore the dynamics of these systems using the movie code. The differences are stark, and range from flocking to seemingly senseless flickering in direction.

Challenge Problem: Coherence and Critical Levels of Noise

At what level of noise η does the final value of the order parameter drop nearly to 0? In doing so, use a density, e.g., $N = 300$, $r = 0.3$, and a range of η from 0 to 5.

Solutions to Challenge Problem: Coherence and Critical Levels of Noise. Suppose we vary the strength of the noise, η between 0 to 5. Run the dynamics for 1000 time steps and report the order parameter averaged over the next 1000 time steps. The result is as follows:



Part of the challenge here is that the order parameter can fluctuate as collective flocks emerge and come apart. Visually, we see that the order parameter begins to drop nearly to 0 when $\eta \approx 3$.

When phase transitions occur, order parameters follow a scaling that depends on the microscopic dynamics. Vicsek et al estimate the critical noise value as $\eta_c = 2.9$. This is the noise level below which the boids begin to flock based on the order parameter, $\bar{v}$. If you have the time, re-scale the previous plot and plot with logarithmic axes to show

the power law behavior of the order parameter near the critical point. That is, see if the data you have follows:

$$v_\alpha \sim (\eta_c - \eta)^\beta$$

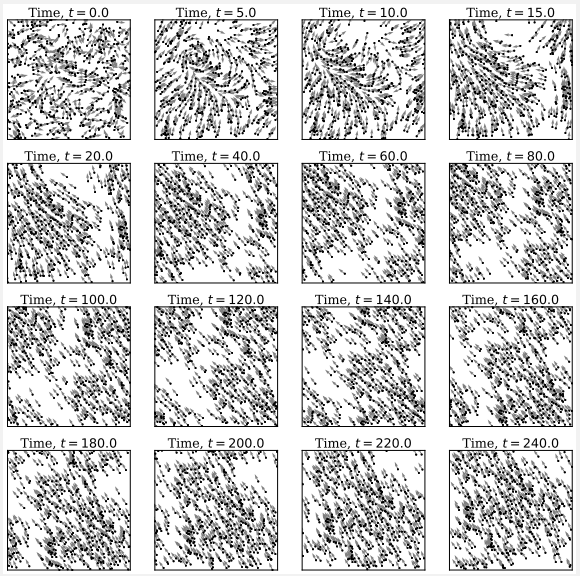
where β shows the scaling of the order parameter near the phase transition.

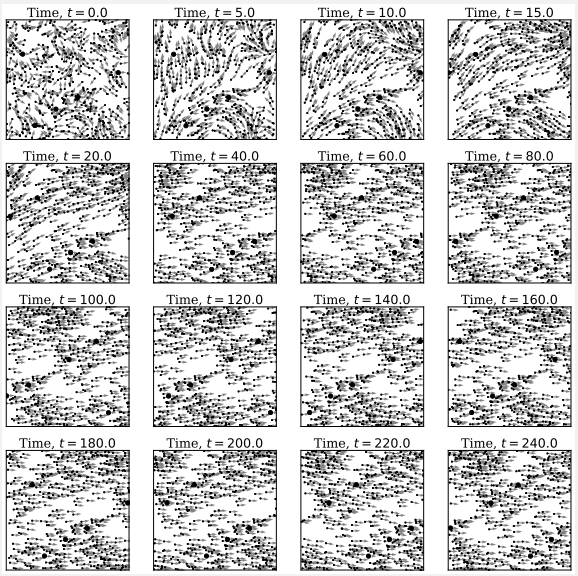
IV. SPP AND THE POWER OF LEADERSHIP

If you implement this quickly, then feel free to try and designate a few "informed" boids who have a preferred direction and see how many it takes to get the others to follow their lead. For example, consider the case of a fixed set m of boids, $i = 1 \dots m$ who all have the same preference for their random direction. This bonus challenge requires that you modify your core code to include informed individuals who always want to go move in preferred direction, albeit with noise.

Challenge Problem: Follow the Leader(s)

Develop a modified SPP model with a flexible number of informed individuals with fixed preferences for direction. As a specific case, try $m = 5$ informed individuals each with preference of $\theta = 0$. If your code works, you should be able to use the same random seed and see the differences in outcome, e.g., for $r = 0.5$, $\eta = 0.25$, and $N = 500$ the results contrast between flocks that go in arbitrary directions (left) and those that go in the preferred direction (right) despite the fact that only 1 in 100 individuals have a preference!





Solutions to Challenge Problem: Follow the Leader(s) The following code includes a new component to the dynamics, individuals with fixed preferences on where to go. In doing so there are two new variables, m, the number of informed individuals, and $\{\theta_m\}$ the set of informed directions. Many variants are possible. Here the model assumes the preferences are fixed, subject to noise. As is apparent in the challenge problem, although flocking occurs without informed leaders, a very small number of leaders can 'steer' the entire flock reliably to their preferred direction.

```
def boids_dyn_leader(pars):
    N=pars['N']
    eta=pars['eta']
    L=pars['L']
    totsteps=pars['totsteps']
    delt=pars['dt']
    v0=pars['v0']
    r=pars['r']
    thetam=pars['thetam']
    m=pars['m']
    #initial conditions
    currx = L*np.random.uniform(size=N)
    curry = L*np.random.uniform(size=N)
    currtheta = 2*np.pi*np.random.uniform(size=N)

    #run the sim
    xt = np.zeros((totsteps,N))
    yt = np.zeros((totsteps,N))
    thetat = np.zeros((totsteps,N))
    for t in range(totsteps):
        #Preallocate new directions
        newtheta = np.zeros(N)
        newtheta[:m+1]=thetam
        # Loop i through each boid and determine new directions
        for i in np.arange(m+1,N):
            # find the boids within R of i
            dx = np.minimum(np.abs(currx-currx[i]),
                            np.abs(currx-(currx[i]+L)),
                            np.abs(currx-(currx[i]-L)))
            dy = np.minimum(np.abs(curry-curry[i]),
                            np.abs(curry-(curry[i]+L)),
                            np.abs(curry-(curry[i]-L)))
            boid_r2 = np.sqrt( (dx)**2 + dy**2 )
            #calculate the new direction of boid i based on the directions of neighbors
            avgthetas = currtheta[boid_r2<=r]
            #average x and y components of vectors
            avgx = np.mean(np.cos(avgthetas))
            avgy = np.mean(np.sin(avgthetas))
            #obtain average direction from arctan, correcting when x direction is negative
            newtheta[i] = np.arctan2(avgy,avgx)
            # end Loop i
        #obtain vector of noise to apply to boid directions
        deltheta = -eta/2+eta*np.random.uniform(size=N)
        #update directions
        currtheta = newtheta+deltheta
        #update positions based on ballistic motion and new directions
        currx = np.mod(currx + v0*np.cos(currtheta)*delt, L)
        curry = np.mod(curry + v0*np.sin(currtheta)*delt, L)

        #track the updates over time
        xt[t,:] = currx
        yt[t,:] = curry
        thetat[t,:] = currtheta

    return [xt,yt,thetat]
```

This last challenge also directly relates to models in which individuals balance their personal preference and the local group preference. Such models can be tuned across a range of weightings and subsets, including leading to unusual outcomes where too many informed leaders leads to indecision such that the group cannot converge on single direction. For those who cook, the old adage too many cooks in the kitchens comes to mind. The homework develops these ideas further, helping you link computational models to the emergent behavior of flocks and groups.